



Data Recovery System



OUTLINE

- ✓ Nested RAID Levels
- ✓ Log-Based Recovery
- ✓ Deferred Database modification technique
- ✓ Immediate Database Modification Technique



Nested RAID Levels

❑ Nested RAID Levels

- Standard RAID drives are built by combining multiple Hard Disk Drives (HDDs) to offer redundancy, better read/write performance, and protection against data loss.
- These RAID drives are built on unique data storage mechanisms, including Striping, Mirroring, and Parity.
- Nested RAID drives are built with two or more of the standard RAID levels (based on a combination of RAID levels).
- They are also called hybrid RAID.
- Nested RAID levels are now gaining popularity for their advantages over standard RAID arrays.
- The standard RAID drives are numbered using a single number like RAID-0, RAID-1, and RAID-5. On the contrary, the Nested RAID levels are numbered using a series of numbers, including RAID-01, RAID-10, RAID-50, RAID-60, and RAID-100.



Nested RAID Levels

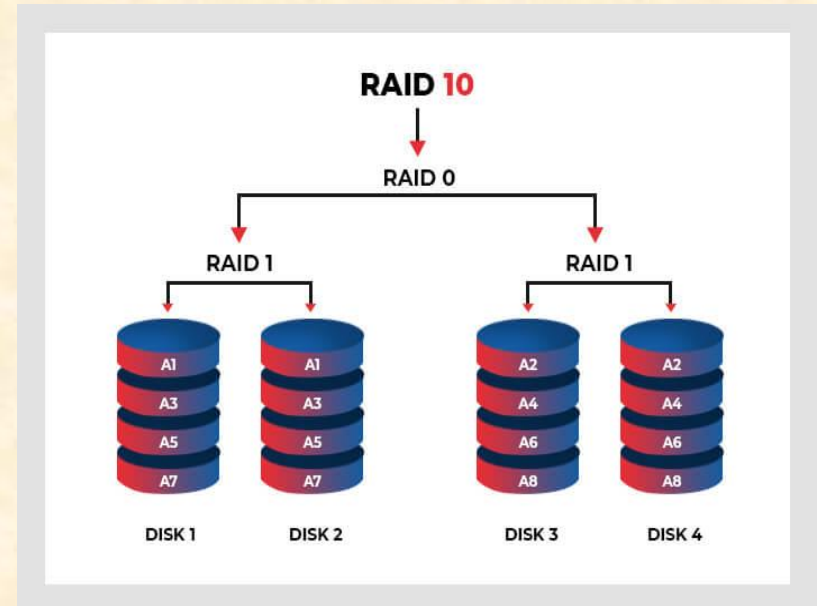
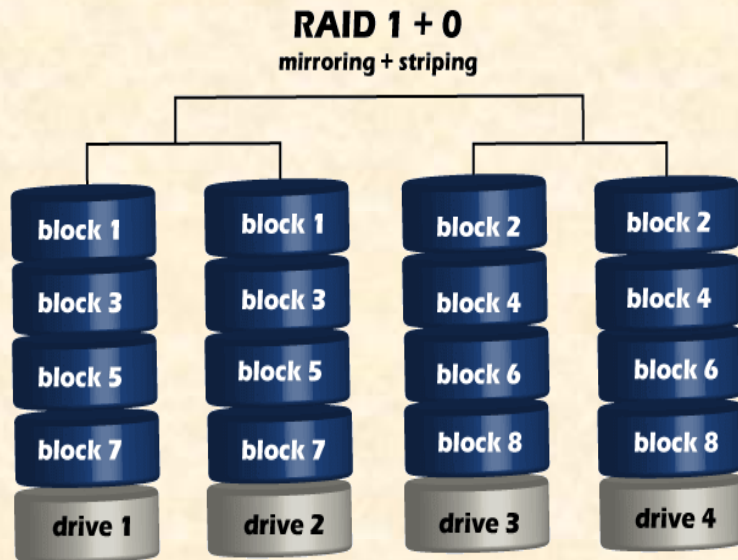
❑ RAID-10 or RAID-1+0

- This combines RAID-1 and RAID-0 in a single system, which offers higher performance than RAID-1, but at a much higher cost.
- This is a nested or hybrid RAID configuration. It provides security by mirroring all data on secondary drives while using striping across each set of drives to speed up data transfers.
- It is also be called a "Mirrored RAID-0 level" or RAID-1+0.
- It can survive up to 2 disks failure, one from either side.
- It requires a minimum of 4 disks.



Nested RAID Levels

❑ RAID-10 or RAID-1+0



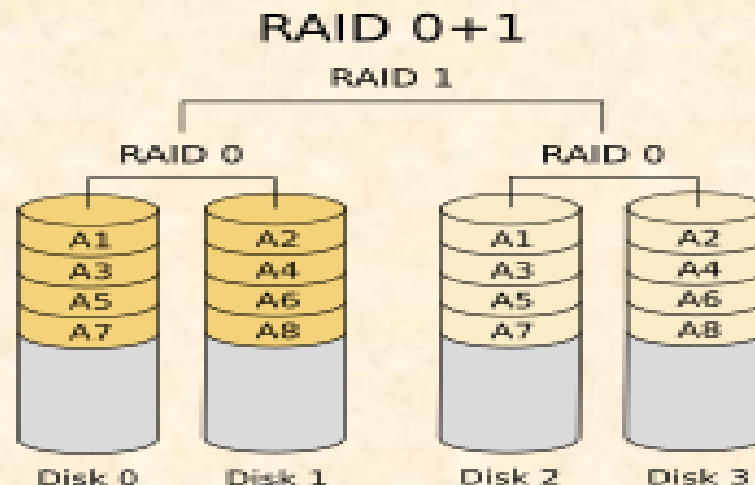
- The total storage is halved due to mirroring, meaning that if you use 4 1-TB drives while setting up RAID-10, it will eventually offer you 2-TB usable storage.



Nested RAID Levels

❑ RAID-01 or RAID-0+1

- RAID-0+1 is similar to RAID-1+0, except the data organization method is slightly different.
- Rather than creating a mirror and then striping the mirror, RAID-0+1 creates a stripe set and then mirrors the stripe set.
- The usable capacity of a RAID-01 array is the same as in a RAID 10.
- At least four disks are required in a standard RAID 01 configuration

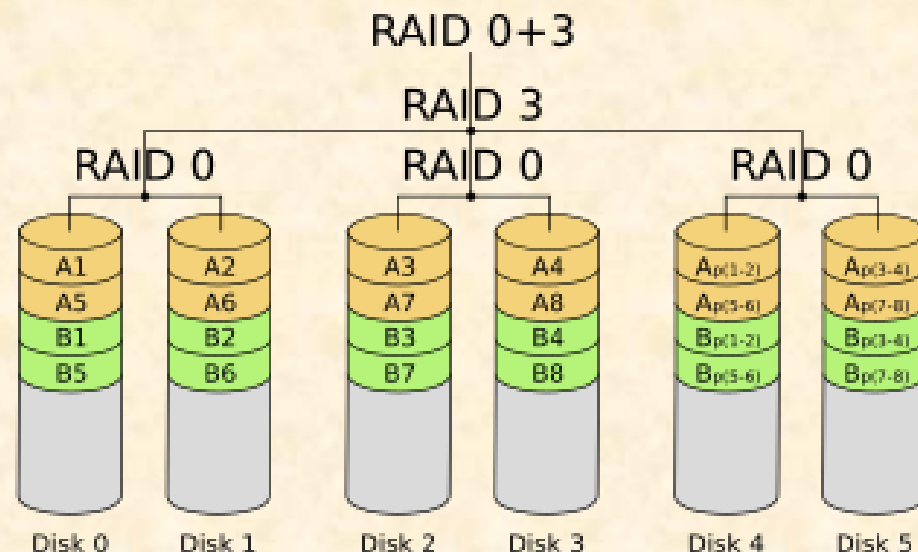


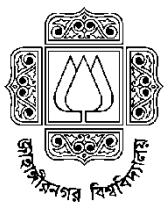


Nested RAID Levels

❑ RAID-03 or RAID-0+3

- Also called **RAID 0+3** and sometimes **RAID 53**.
- This is similar to RAID 01 with the exception that byte-level striping with dedicated parity is used instead of mirroring
- This level uses striping similar to RAID 0 for RAID-3.
- This offers higher performance than RAID-3 but at a higher cost.





Nested RAID Levels

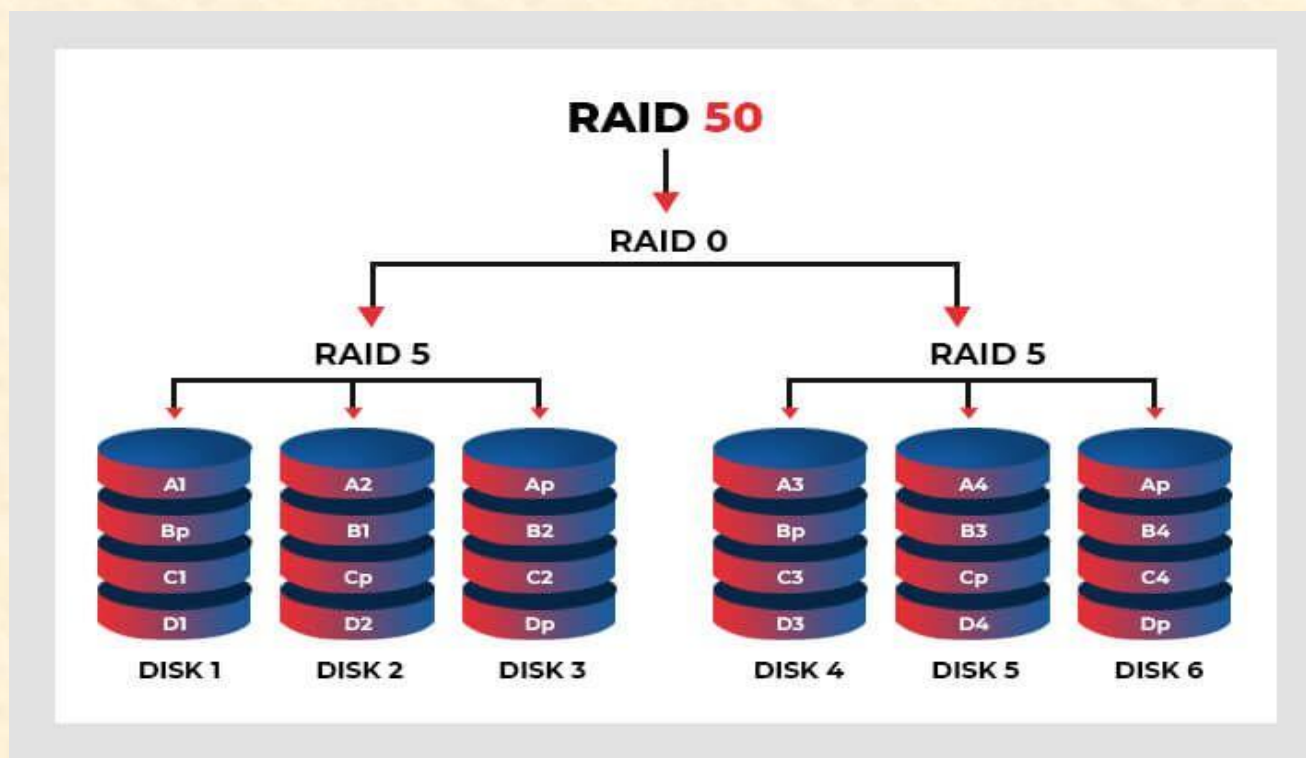
❑ RAID-50 or RAID-5+0

- This configuration combines RAID-5 distributed parity with RAID-0 striping to improve RAID-5 performance without reducing data protection.
- It is also called RAID 5+0.
- This array offers the benefits of both RAID 5 and 0 by following the Parity & Striping mechanism.
- It needs at least 6 disks to set up RAID 50 in a nested environment.
- It can support a maximum of 48 disks in single or multiple mirrored arrays of RAID 5.
- With this Nested RAID level, you can get higher read/write speed performance, faster rebuilds, and better data protection.
- This hybrid level can withstand up to 4 disks failure.



Nested RAID Levels

❑ RAID-50 or RAID-5+0

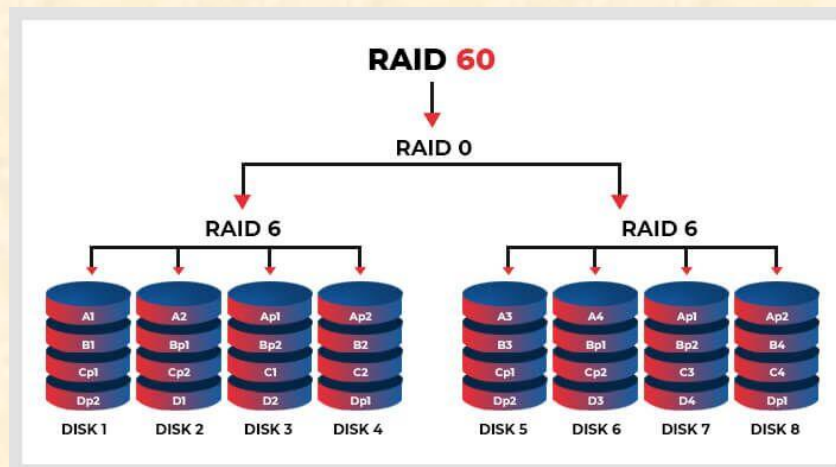




Nested RAID Levels

❑ RAID-60 or RAID-6+0

- It combines the dual parity of RAID-6 and block-level striping of RAID-0 array.
- It requires at least eight disks and this configuration can support up to 128 disks.
- Although it is similar to RAID 50, it offers better redundancy by providing dual drive failure protection.
- However, it has lower write data performance rate.

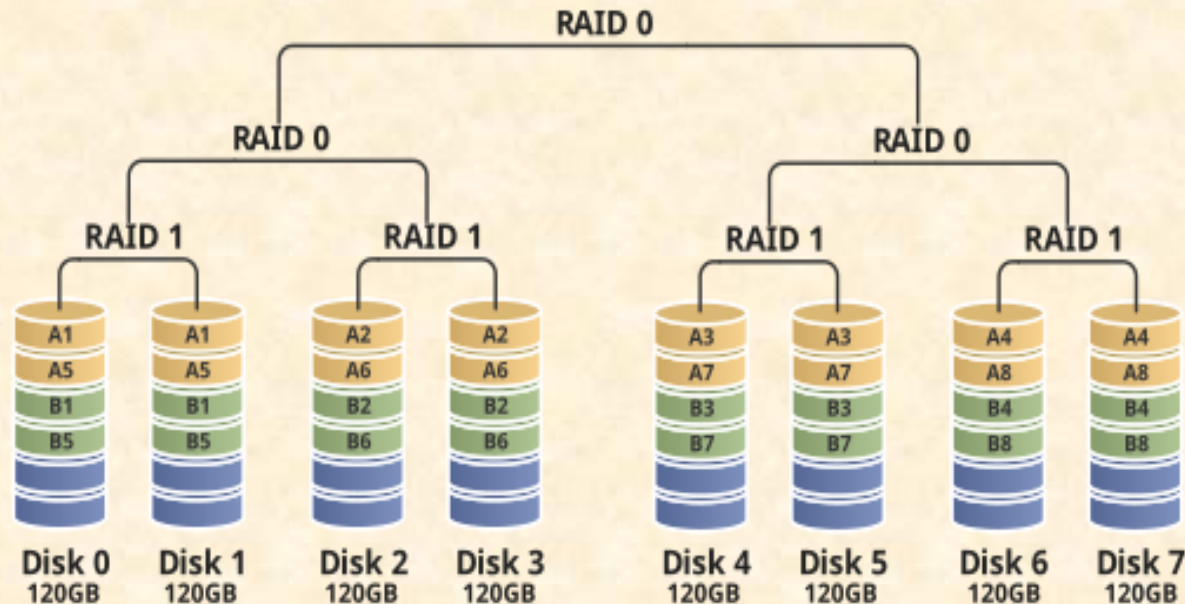




Nested RAID Levels

❑ RAID-100 (RAID-10+0)

- Sometimes called RAID-10+0
- It is a stripe of RAID-10s.
- This is logically equivalent to a wider RAID-10 array, but is generally implemented using software RAID-0 over hardware RAID-10.





Nested RAID Levels

❑ Nested/Hybrid RAID Benefits

- **Improved Performance:** By combining the striping of RAID 0 with the redundancy of other RAID levels, nested RAID can significantly enhance read and write speeds.
- **Enhanced Fault Tolerance:** With multiple levels of redundancy, nested RAID offers better protection against data loss in case of drive failures.
- **Expanded storage capacity:** Depending on the type of nested RAID, you can benefit from a larger capacity than traditional RAID configurations. For example, RAID 10 offers a higher storage capacity than RAID 1 alone.
- **Flexibility:** Nested RAID configurations can be customized to fit specific storage needs, making it an ideal solution for businesses with varying data requirements.



Nested RAID Levels

❑ Nested/Hybrid RAID Limitations

- **Complexity:** Nested RAID can be complicated to set up and manage, requiring high technical expertise.
- **Expensive:** Due to multiple drives, nested RAID can be more costly than traditional RAID configurations.
- **Performance Degradation:** In certain nested RAID configurations, performance can degrade significantly if multiple drives fail.



Nested RAID Levels

❑ Non-Standard RAID Levels

- In some cases- non-standard RAID levels are used.
- Many non-standard RAID levels were developed by individual manufacturers for use in their products, and some are only designed to work with specific system specifications.
- Some non-standard RAID levels:
 - RAID 1E
 - RAID 5E
 - RAID 5EE
 - RAID 6E

E stands for Enhanced

RAID 1E, RAID 5E, RAID 5EE, and RAID 6E refer to variants of RAID 1, 5 and 6.



Nested RAID Levels

❑ Non-Standard RAID Levels

- There are other forms of non-standard RAID:
 - RAID-DP uses double-parity disk protection to help prevent disk errors and data corruption.
 - RAID Z was designed by the Sun Company specifically for use with the ZFS file system
 - MD RAID 10 only works on Linux-based operating systems.
 - Adaptive-RAID This level enables the RAID controller to decide how to store the parity on disks. It will choose between RAID 3 and RAID 5, depending on which RAID set type will perform better with the kind of data being written to the disks.
 - RAID-7 A non-standard RAID level is based on RAID 3 and RAID 4 that adds caching. It includes a real-time embedded OS as a controller, caching via a high-speed bus, and other stand-alone computer characteristics.
- Since manufacturers and developers are free to create their own configurations to meet specialty needs, the possibilities are nearly endless.



Nested RAID Levels

❑ Which is the best RAID level?

- Each RAID level offers different benefits. However, while choosing the best level for RAID systems, you must check for all the features like Storage Capacity, Application Performance, Cost, and so on. So evaluate all these aspects and your requirements, and choose wisely.

❑ Which RAID level is most secure?

- When it comes to the most secure RAID level, RAID-10 is considered the best. This is because RAID-10 stripes data across multiple disks. Therefore, the data chunks can be read or written to multiple disks simultaneously. Additionally, RAID-10 offers mirroring that duplicates all your data, thereby offering faster read-write performance and security.



Log-Based Recovery

❑ What is Log-Based Recovery?

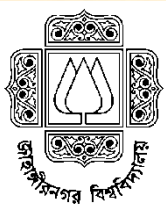
- The log is a sequence of records. Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- If any operation is performed on the database, then it will be recorded in the log.
- The process of storing the logs should be done before the actual transaction is applied in the database.
- There are several types of log records.



Log-Based Recovery

❑ Fields of Log Record

- A log record has the following fields:
 - **Transaction identifier:** Unique identifier that distinguishes one transaction from another.
 - **Data-item identifier:** It uniquely identifies data. Typically, it is the location on disk of the data item.
 - **Old value:** This is the value of the data item before writing
 - **New value:** This is the value of the data item after it has been written.



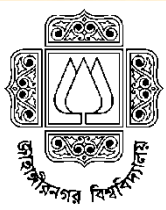
Log-Based Recovery

□ Types of Log Records

- There are various types of log records:

Log Records	Description
<Ti start>	Transaction Ti has started.
<Ti, Xj, V1, V2>	Transaction Ti has performed a write operation on data item Xj. Xj contains value V1 before the write and will contain value V2 after the write.
<Ti commit>	Transaction Ti has committed.
<Ti abort>	Transaction Ti has aborted.

- Whenever a transaction performs a write, a log record is created before the database is modified.
- The logs may or may not record the reading of data items. Because reading a data item has no effect on the consistency of the database.



Log-Based Recovery

❑ Two operations during the recovery process

- **Undo(T_i)**: Restores the old values of each of the data items in T_i that have been updated by the transaction T_i .
- **Redo(T_i)**: Updates all data items updated by transaction T_i with their new values.

❑ Database Modification Techniques for ensuring transaction atomicity

- There are two approaches to modify the database:
 - Deferred database modification and
 - Immediate database modification



Log-Based Recovery

- ❑ **Deferred Database Modification Technique for ensuring transaction atomicity**
 - The deferred modification technique occurs if the transaction does not modify the database until it has committed.
 - In this method, all the logs are created and stored in the stable storage, and the database is updated when a transaction commits.



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

Recovery technique based on deferred update:

- Follow these steps to recover using the deferred update process:
 - Search the log for the most recent checkpoint record. This is the last time the transaction data was sent to disk. All transactions that were started and committed **before the last checkpoint** do not need to do anything because the data has already been saved.
 - For any transaction T, if the log contains both a <T start> record and a <T commit> record that appear in the log **after the last checkpoint**, this method uses the log records to redo the transaction and update the database using new value in the log.
 - For any transaction T, if the log only has a <T start> record but not a <T commit> record **after the last checkpoint**, nothing needs to be done because the database has never been updated.



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

- To illustrate the concept of deferred update process, consider an example involving two transactions T1 and T2. Transaction T1 transfers Tk. 200 from account A to account B and transaction T2 withdraws Tk. 100 from account C. The initial balances of accounts A, B and C are Tk. 1000, Tk. 1500 and Tk. 2000 respectively. Suppose transactions T1 and T2 are executed sequentially with T1 followed by T2.

T1	T2
Read (A) $A := A - 200$ Write (A) Read (B) $B := B + 200$ Write (B)	Read (C) $C := C - 100$ Write (C)



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

– The order in which these transactions will be stored in log:

- When a transaction starts, a transaction start record is written in the log. Here when transaction T1 starts, a start record $\langle T1 \text{ start} \rangle$ is written in the log. Similarly, when a transaction T2 starts, a start record $\langle T2 \text{ start} \rangle$ is written in the log.
- When a write operation is performed, a write log record is written. Here when transaction T1 performs a Write (A) operation, a write record $\langle T1, A, 800 \rangle$ is written in the log. Similarly on execution of Write (B) operation by transaction T1 and Write(C) operation by transaction T2, appropriate write log records are written. One should note at this point that updates are not written to disk.



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

– The order in which these transactions will be stored in log:

- When the transaction is committed, a commit record is written in the log. Here when transaction T1 commits, a transaction commit record <T1 commit> is written to the log. Similarly, when transaction T2 commits, a transaction commit record <T2 commit> is written to the log.



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

- The full contents of the log and the database when these two transactions (T1 and T2) are executed serially with T1 followed by T2 are shown below:

Log	Database
<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit>	A=800 B=1700
<T2 start> <T2, C, 1900> <T2 commit>	C=1900



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

- In case of failure, the log files are examined starting with the last entry till the most recent checkpoint. The recovery scheme uses the following operation:
 - $\text{redo}(T_i)$ sets the value of all data items updated by transaction T_i to the new values.
- After a failure, the recovery scheme consults the log to determine which transactions need to be redone. Transaction T_i needs to be redone if and only if the log contains both the record $\langle T_i \text{ start} \rangle$ and the record $\langle T_i \text{ commit} \rangle$. If the log contains only the start record $\langle T_i \text{ start} \rangle$ but no commit record $\langle T_i \text{ commit} \rangle$, Transaction T_i needs to be restarted/re-executed.

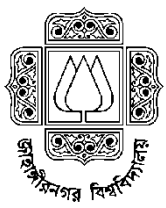


Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

- As an illustration, consider the following log records for transactions T1 and T2 at three different times:

Condition 1	Condition 2	Condition 3
<T1 start> <T1, A, 800> <T1, B, 1700>	<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit> <T2 start> <T2, C, 1900>	<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit> <T2 start> <T2, C, 1900> <T2 commit>
Crash	Crash	Crash



Log-Based Recovery

❑ Deferred Database Modification Technique for ensuring transaction atomicity

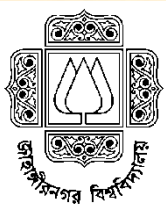
Condition 1	Condition 2	Condition 3
Whenever the recovery process starts, no redo action needs to be taken because no commit record appears in the log and values of account A, B and C remain same i.e. 1000, 1500 and 2000 respectively.	Whenever the recovery process starts, it finds that transaction T1 is already committed in the log. So operation Redo for transaction T1 must be performed and all data items updated by transaction T1 are copied into database (i.e. A = 800 and B = 1700). But the transaction T2 is not yet committed in the log. So there is no need to redo the transaction T2. This transaction T2 is deleted from log and re-executed. So value of data item C = 2000.	Whenever the recovery process starts it finds that there are committed records in the log for both transactions T1 and T2. This results in redo operations being performed for both T1 and T2. Hence the value of data items A, B and C is 800, 1700 and 1900 respectively.
No Redo	Redo (T1) T2 must be re-executed	Redo (T1) , Redo (T2)
A = 1000, B = 1500, C = 2000	A = 800, B = 1700, C = 2000	A = 800, B = 1700, C = 1900



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- The Immediate modification technique applies if database modification occurs while the transaction is still active.
- In this technique, the database is modified immediately after every operation. It follows an actual database modification.



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

Recovery based on immediate update:

- In this technique, when a transaction issues an update command, the database can be updated immediately without waiting for the commit record.
- The immediate update technique for recovery is also known as backward recovery because it tracks backward through the log and re-rolls all data items to their previous values.
- In the event of a crash, the system must use the old-value field of the log records to restore the modified data items to the value they had prior to the start of the transaction. The undo operation accomplishes this restoration.



Log-Based Recovery

- ❑ Immediate Database Modification Technique for ensuring transaction atomicity

Undo/Redo recovery technique based on immediate update:

- If a transaction fails after updating some changes in the database but before reaching its commit record, the effect of its operation on the database must be **undone** i.e. transaction must be rolled back.
- If the transaction fails after reaching its commit record, it is necessary to perform the **redo** operation.



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

Recovery steps using the immediate update process:

- Search the log for the most recent checkpoint record. This is the last time the transaction data was sent to disk. All transactions that were started and committed **before the last checkpoint** do not need to do anything because the data has already been saved.
- For any transaction T, if the log contains both a <T start> record and a <T commit> record that appear in the log **after the last checkpoint**, this method uses log records to **redo** the transaction and update the database using the new value in the transaction log.
- For any transaction T, if the log contains only <T start> record but no <T commit> record **after the last checkpoint**, this method uses the log records to **undo** the operations using the old values in the log. To undo multiple write operations from the log we must precede in the reverse order in which they were written.



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- To illustrate the concept of immediate update process, consider an example involving two transactions T1 and T2. Transaction T1 transfers Tk. 200 from account A to account B and transaction T2 withdraws Tk. 100 from account C. The initial balances of accounts A, B and C are Tk. 1000, Tk. 1500 and Tk. 2000 respectively. Suppose transactions T1 and T2 are executed sequentially with T1 followed by T2.

T1	T2
Read (A) $A := A - 200$ Write (A) Read (B) $B := B + 200$ Write (B)	Read (C) $C := C - 100$ Write (C)



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- The full contents of the log and the database when these two transactions (T1 and T2) are executed serially with T1 followed by T2 are shown below:

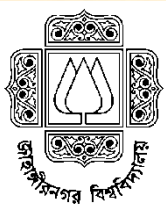
Log	Database
<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit>	A=800 B=1700
<T2 start> <T2, C, 2000, 1900> <T2 commit>	C=1900



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- In case of failure, the log files are examined starting with the last entry till the most recent checkpoint. The recovery scheme uses the following operations:
 - $\text{undo}(T_i)$ restores the value of all data items updated by transaction T_i to the old values.
 - $\text{redo}(T_i)$ sets the value of all data items updated by transaction T_i to the new values.



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- The set of data items updated by T_i and their respective old values can be found in the log. After a failure has occurred, the recovery scheme consults the log to determine which transactions need to be redone, and which need to be undone:
 - Transaction T_i needs to be undone if the log contains the record $\langle T_i \text{ start} \rangle$, but does not contain the record $\langle T_i \text{ commit} \rangle$.
 - Transaction T_i needs to be redone if the log contains both the record $\langle T_i \text{ start} \rangle$ and the record $\langle T_i \text{ commit} \rangle$.



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

- As an illustration, we consider the following three cases. The log records for each of these cases appear in the following Figure.

Condition 1	Condition 2	Condition 3
<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700>	<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit> <T2 start> <T2, C, 2000, 1900>	<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit> <T2 start> <T2, C, 2000, 1900> <T2 commit>
Crash	Crash	Crash



Log-Based Recovery

❑ Immediate Database Modification Technique for ensuring transaction atomicity

Condition 1	Condition 2	Condition 3
Whenever the recovery process starts, Transaction T1 should be undone. Because no commit record appears in the log. So old values are copied to database.	Whenever the recovery process starts, transaction T1 must be redone and transaction T2 must be undone. Updates made by transaction T1 are copied to database but transaction T2 has not yet committed in the log record, so it must be undone.	Whenever the recovery process starts, both transactions T1 and T2 should be redone as commit records appear in the log. So updates made by them are copied to database.
Undo (T1)	Redo (T1), Undo (T2)	Redo (T1) , Redo (T2)
A = 1000, B = 1500, C = 2000	A = 800, B = 1700, C = 2000	A = 800, B = 1700, C = 1900



Log-Based Recovery

❑ Difference between Deferred update and Immediate update

Deferred Update:

- It is a technique for the maintenance of the transaction log files of the DBMS.
- It is also called NO-UNDO/REDO technique.
- It is used for the recovery of transaction failures that occur due to power, memory, or OS failures.
- Whenever any transaction is executed, the updates are not made immediately to the database. They are first recorded on the log file and then those changes are applied, once the commit is done. This is called the “Re-doing” process.
- Once the rollback is done, none of the changes are applied to the database and the changes in the log file are also discarded.
- If the commit is done before crashing the system, then after restarting the system the changes that have been recorded in the log file are thus applied to the database.



Log-Based Recovery

❑ Difference between Deferred update and Immediate update

Immediate Update:

- It is a technique for the maintenance of the transaction log files of the DBMS.
- It is also called UNDO/REDO technique.
- It is used for the recovery of transaction failures that occur due to power, memory, or OS failures.
- Whenever any transaction is executed, the updates are made directly to the database and the log file is also maintained which contains both old and new values. Once the commit is done, all the changes get stored permanently in the database, and records in the log file are thus discarded.
- Once rollback is done the old values get restored in the database and all the changes made to the database are also discarded. This is called the “Un-doing” process.
- If the commit is done before crashing the system, then after restarting the system the changes are stored permanently in the database.



Checkpoints in DBMS

❑ What is a Checkpoint in DBMS?

- A Checkpoint is used to declare a point before which the DBMS was in a consistent state, and all transactions were committed.
- During transaction execution, such checkpoints are traced. Upon reaching the checkpoint (save point), the log file is destroyed by saving its update to the database. Then a new log is created with upcoming execution operations of the transaction and it will be updated until the next checkpoint and the process continues.



Checkpoints in DBMS

❑ Why do we need Checkpoints?

- In case of system failures, we check the logs to determine which of the transactions need to be redone and those which need to be undone. For this, we search the entire log to determine this information which results in two major difficulties:
 - It is very time consuming as we may have to search all log files.
 - Most of the transactions which need to be redone have already written their updates to the database.
 - **For Example:** In an online database system such as airline reservation system, hundreds of transactions could be handled per minute, so for such type of databases the log contains a large volume of information.
 - Therefore, a scheme called checkpoint is used to limit the volume of log information that has to be handled and processed in case of system failure.
- Checkpoints are utilized for removing all previous transaction logs and storing them in permanent storage.
- When a checkpoint occurs, the DBMS will write a copy of the current state of the database to disk.



Checkpoints in DBMS

❑ Types of Checkpoints

- There are basically two types of Checkpoints:
 - Automatic Checkpoint
 - Manual Checkpoint

Automatic Checkpoint:

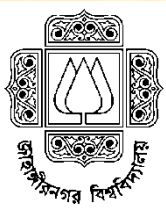
- These checkpoints occur at regular intervals, such as every hour or every day.
- These intervals are set by the database administrator.
- They are generally used by heavy databases as they are frequently updated, and we can recover the data easily in case of failure.
- For example, in SQL Server, the default interval for automatic checkpoints is every minute, but this can be configured to occur at different intervals.



Checkpoints in DBMS

❑ Manual Checkpoint

- These are the checkpoints that are manually set by the database administrator.
- Manual checkpoints are generally used for smaller databases.
- They are updated very less frequently only when they are set by the database administrator.



Checkpoints in DBMS

❑ Advantages of Checkpoints

- Checkpoints help us in recovering the transaction of the database in case of failure.
- It enhances the consistency of the database in case when multiple transactions are executing in the database simultaneously.
- Checkpoints are used to prevent unnecessary redo operations.
- It has a very low overhead and can be done frequently.

❑ Disadvantages of Checkpoints

- Checkpoints can only be used to restore from logical errors.
- Because all the data blocks are on the same physical device, checkpoints cannot be used to restore files due to a media failure.



Checkpoints in DBMS

❑ Applications of Checkpoints

- **Backup and Recovery** – Checkpoints are used to ensure that the database can be recovered quickly in the event of a failure. It ensures that data is not lost in the event of a failure.
- **Performance Optimization** – Checkpoints can also be used to improve performance by reducing the amount of work that needs to be done during recovery.
- **Auditing** – Checkpoints can be used to track changes to the database over time, which is useful for auditing purposes.



Checkpoints in DBMS

❑ **What are the operations performed, when a system issues a checkpoint?**

- When a system issues a checkpoint, the following operations are performed:
 - The log information from the buffers in the volatile storage is copied to log on the stable storage.
 - All database updates from the buffers in the volatile storage are written to the disk.
 - Write a checkpoint record to log on the disk, so subsequent database recovery coordinates with the checkpoint.



Checkpoints in DBMS

- ❑ **Recovery using Checkpoints (Discuss how to recover the database from failure using Checkpoints)**
 - In the following manner, a recovery system recovers the database from this failure:

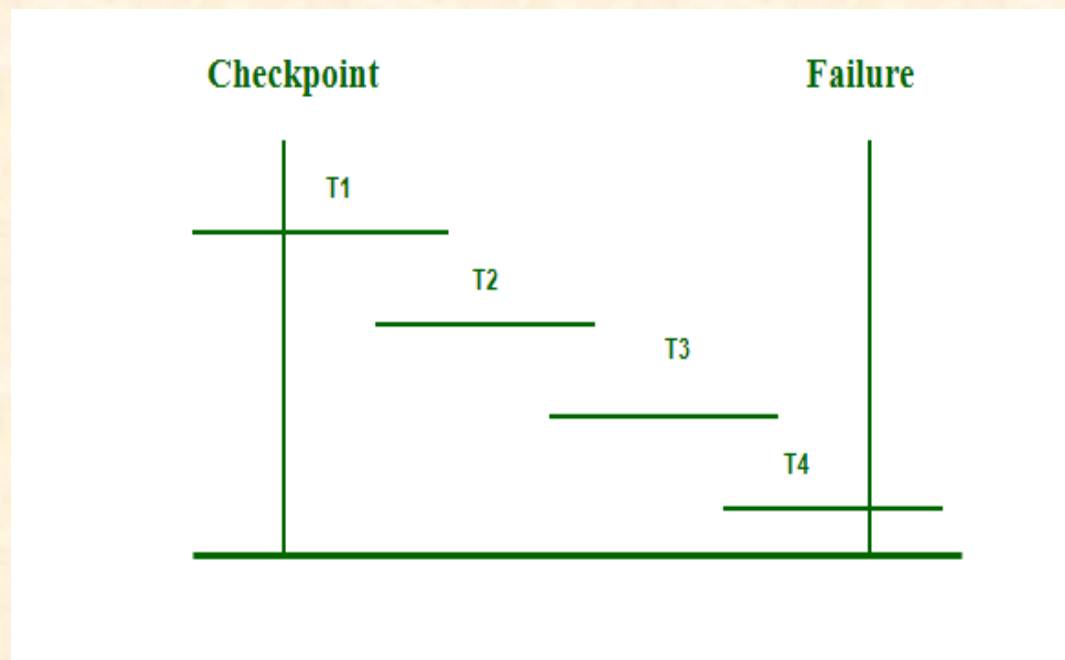


Fig. Checkpoints in DBMS



Checkpoints in DBMS

❑ Recovery using Checkpoints

- The recovery system reads the logs backward from the end to the last checkpoint i.e. from T4 to T1.
- It maintains two lists – a undo-list and a redo-list.
- Whenever the recovery system sees a log with $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$ or only $\langle T_n, \text{commit} \rangle$, it will put that transaction in redo-list. In the above figure, T2 and T3 contain $\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$ whereas T1 will have only $\langle T_n, \text{commit} \rangle$. Hence, T1, T2, and T3 are in the redo-list.
- Whenever the recovery system sees a log with $\langle T_n, \text{start} \rangle$ but no commit, it will put that transaction in undo-list. In the above figure, T4 has only $\langle T_n, \text{start} \rangle$ but no $\langle T_n, \text{commit} \rangle$. Hence, T4 will be put on the undo-list.
- All the transactions in the redo-list are deleted with their previous logs and then redone before saving their logs.
- All the transactions in the undo-list are undone and their logs are deleted.